

Core & Advanced Java

Complete Beginner Guide with Examples & Diagrams

Interview Preparation for Java Backend Developer Roles

Java 8+

Collections

Concurrency

JVM Memory

Infosys Ready

15 Topics · Detailed Explanations · Code Examples · Interview Q&A

Table of Contents

- | | |
|--|--------------------------------|
| 1. == vs equals() | 9. volatile Keyword |
| 2. hashCode() and equals() Consistency | 10. synchronized Keyword |
| 3. String vs StringBuilder vs StringBuffer | 11. Deadlock |
| 4. Immutability | 12. Java Memory Model |
| 5. Abstract Class vs Interface | 13. Heap vs Stack vs Metaspace |
| 6. Overloading vs Overriding | 14. Garbage Collection |
| 7. HashMap Internals | 15. LRU Cache Design |
| 8. ArrayList vs LinkedList | 16. Quick Revision Cheat Sheet |

1. == vs equals()

In Simple Terms

`==` checks whether two variables point to the **same object in memory** (same address). `equals()` checks whether two objects have the **same content or meaning** — but only when the class has overridden `equals()`. For objects, `==` almost always gives the wrong answer when you want to compare values.

Visual Diagram

Memory Layout:

```
variable a → [ String object: "Hello" ]    address: 0x100
variable b → [ String object: "Hello" ]    address: 0x200 (different object, same text)
variable c → same as a (address 0x100)

a == c      → TRUE   (both point to same memory address)
a == b      → FALSE  (different memory addresses)
a.equals(b) → TRUE   (same character content – String overrides equals())
```

Detailed Explanation

When you write `new String("Hello")` twice, Java creates **two separate objects** on the heap, even if the text is identical. The `==` operator compares references (memory addresses), so it returns `false`.

The `String` class overrides `equals()` to compare characters one by one. That is why `a.equals(b)` returns `true` when both contain "Hello".

For **primitive types** (`int`, `boolean`, `char`, `double`), there is no `equals()` method. The `==` operator compares actual values directly — and that is the correct way to compare primitives.

Special Case: Integer Caching

Java caches `Integer` objects from **-128 to 127**. So `Integer a = 100; Integer b = 100; a == b` is `true` because both reference the same cached object. But `Integer x = 200; Integer y = 200; x == y` is `false` because 200 is outside the cache range and two new objects are created.

Code Example

```
String a = new String("Hello");
String b = new String("Hello");
String c = a;

System.out.println(a == b);    // false – different objects in memory
System.out.println(a.equals(b)); // true  – same text content
System.out.println(a == c);    // true  – c points to same object as a

// Primitives – == compares values directly
int x = 10, y = 10;
System.out.println(x == y);    // true

// Integer caching trap
Integer p = 100, q = 100;
System.out.println(p == q);    // true  (cached range -128 to 127)

Integer m = 200, n = 200;
System.out.println(m == n);    // false (outside cache – new objects)
```

Type	Use == for	Use equals() for
Objects (String, Employee, etc.)	Checking if same reference only	Comparing logical/value equality
Primitives (int, boolean, etc.)	Comparing values	Not applicable

Interview Question: What is the difference between == and equals()?

30-second answer: == compares memory references (same object?). equals() compares content (same value?). For objects always use equals() unless you specifically need to check if two variables point to the exact same object.

Detailed answer: == is an operator that compares references for objects and values for primitives. equals() is a method defined in Object class that compares content — but many classes like String override it. Default Object.equals() actually behaves like ==. Always override equals() in your custom classes when logical equality matters.

2. hashCode() and equals() Consistency

The Contract — Must Memorize

1. If `a.equals(b)` is **true**, then `a.hashCode() == b.hashCode()` **MUST** be true.
2. If two objects have the same hashCode, `equals()` does **not** have to be true (hash collision is allowed).
3. If you override `equals()`, you **must** override `hashCode()` too.

Real-Life Analogy

Think of a library with numbered shelves. `hashCode()` tells you the shelf number. `equals()` checks if the book on that shelf is the exact one you want. If two books are the same (equals), they **MUST** be on the same shelf (same hashCode). If you put equal books on different shelves, you will never find them — that is the bug that happens in HashMap when you break this contract.

Why It Matters for HashMap and HashSet

HashMap lookup process:

- Step 1: `key.hashCode()` → calculate bucket index (which shelf?)
- Step 2: Go to that bucket
- Step 3: Compare each key in bucket using `equals()`

If `equals()` says two objects are equal BUT `hashCode()` differs:

- HashMap looks in the **WRONG** bucket
- `get()` returns null even though the data **EXISTS**
- Silent, extremely hard-to-debug production bug!

Code Example — Correct Implementation

```
class Employee {
    private int id;
    private String name;

    public Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Employee e)) return false;
        return id == e.id; // same id means same employee
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(id); // MUST use same field as equals()!
    }
}

// Usage in HashSet
Set<Employee> employees = new HashSet<>();
employees.add(new Employee(101, "Ashish"));
System.out.println(employees.contains(new Employee(101, "Ravi"))); // true - same id
```

Common Mistakes to Avoid

- Overriding `equals()` but forgetting `hashCode()`
- Using `id` in `equals()` but `name` in `hashCode()` — they must use the same fields
- Changing a field used in `hashCode()` after putting the object in a `HashMap` — the object becomes "lost"
- Not handling null in `equals()` — always check `instanceof` first

Interview Question: Why must `hashCode()` be consistent with `equals()`?

30-second answer: `HashMap` uses `hashCode` to find the bucket and `equals` to find the exact key. If equal objects have different hash codes, `HashMap` looks in the wrong bucket and returns null — data appears lost.

3. String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutable?	No — immutable	Yes — mutable	Yes — mutable
Thread-safe?	Yes (because immutable)	No	Yes (synchronized methods)
Speed	Slow for many concatenations	Fast	Slower than StringBuilder
Introduced	Java 1.0	Java 5	Java 1.0
When to use	Constants, keys, sharing between threads	Building strings in loops (single thread)	Multi-thread string building (rare today)

Analogy

String = writing on stone. To change it, you must carve a new stone. **StringBuilder** = writing on a whiteboard. You can erase and rewrite on the same board. **StringBuffer** = a whiteboard with a lock — only one person can write at a time.

Why String Concatenation in Loops is Slow

String concatenation in a loop (BAD):

```
s = ""
s = s + "A" → creates NEW String object "A"    (old "" discarded)
s = s + "B" → creates NEW String object "AB"   (old "A" discarded)
s = s + "C" → creates NEW String object "ABC"  (old "AB" discarded)
```

10,000 iterations = 10,000 String objects created → slow + memory pressure + GC overhead

StringBuilder in a loop (GOOD):

```
sb = [ empty buffer ]
sb.append("A") → modifies SAME buffer: "A"
sb.append("B") → same buffer: "AB"
sb.append("C") → same buffer: "ABC"
```

10,000 iterations = 1 object modified → fast and efficient

Code Example

```
// BAD – creates thousands of String objects
String result = "";
for (int i = 0; i < 10000; i++) {
    result = result + i;
}

// GOOD – one mutable buffer, very fast
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) {
    sb.append(i);
}
String result = sb.toString();

// StringBuffer – same API, but thread-safe (synchronized)
StringBuffer sbf = new StringBuffer("Hello");
sbf.append(" World");
```

Interview Tip

In 95% of real projects, use **StringBuilder** for string building. StringBuffer is rarely needed because each thread typically has its own StringBuilder. String is perfect for constants, configuration keys, and HashMap keys.

4. Immutability

In Simple Terms

An **immutable object** cannot be changed after it is created. Any operation that appears to modify it actually creates a **brand new object**. The original object remains completely untouched. Java's `String`, `Integer`, and `LocalDate` are all immutable.

Diagram — How String Immutability Works

```
String name = "Ashish"

name → [ A | s | h | i | s | h ] (fixed in memory, cannot change)

name.concat(" Shah")
    |
    → Creates NEW object: [ A | s | h | i | s | h | | S | h | a | h ]
      (original "Ashish" object still exists unchanged)

System.out.println(name); // still prints "Ashish"
// name still points to original object!
```

Why Immutability Matters

- **Thread-safe** — no thread can modify it, so no synchronization needed
- **Safe as HashMap keys** — hashCode never changes because the object never changes
- **Predictable** — easier to debug; you know the value will not change unexpectedly
- **Security** — sensitive data like passwords cannot be altered after creation
- **Can be cached/shared** — safe to share between threads without copying

How to Create Your Own Immutable Class

```
// Four rules for immutability:
public final class ImmutablePerson {           // 1. final class – cannot subclass

    private final String name;                 // 2. private final fields
    private final List<String> hobbies;

    public ImmutablePerson(String name, List<String> hobbies) {
        this.name = name;
        this.hobbies = List.copyOf(hobbies); // 3. defensive copy of mutable inputs
    }

    public String getName() { return name; }
    public List<String> getHobbies() { return hobbies; } // unmodifiable
    // 4. NO setters – no way to change state after construction
}

// Java 17 Record – immutable by design (auto-generates constructor, equals, hashCode)
public record User(Long id, String email) {}
```

5. Abstract Class vs Interface

Feature	Abstract Class	Interface
What is it?	An incomplete class — can have both abstract and concrete methods	A contract that defines what a class must do
Inheritance	<code>extends</code> — only ONE class allowed	<code>implements</code> — MANY interfaces allowed
Fields	Can have any instance fields and constructors	Only <code>public static final</code> constants (before Java 8)
Constructor	Yes — can have constructors	No — interfaces are not instantiated
Methods	Abstract methods + concrete methods with body	Abstract + default/static methods (Java 8+)
When to use	Shared code + strong "is-a" relationship (Dog IS AN Animal)	Capability or role ("can fly", "can swim")

Analogy

Abstract class = a partially built house with some rooms finished. Child classes complete the remaining rooms but inherit the finished ones. **Interface** = a job description listing skills required. A person can have multiple job descriptions (implement multiple interfaces) but can only live in one house (extend one class).

Diagram

```
graph TD
    Animal["Animal (abstract class)"]
    Dog["Dog extends Animal"]
    Cat["Cat extends Animal"]
    Flyable["Flyable (interface)"]
    Swimmable["Swimmable (interface)"]
    Duck["Duck implements Flyable, Swimmable  
(one class + multiple interfaces)"]

    Animal --> Dog
    Animal --> Cat
    Flyable --> Duck
    Swimmable --> Duck
```

Animal (abstract class)

abstract sound() breathe() { concrete code shared by all }

Dog extends Animal Cat extends Animal

Flyable (interface) Swimmable (interface)

Duck implements Flyable, Swimmable
(one class + multiple interfaces)

Code Example

```
abstract class Animal {
    protected String name;
    Animal(String name) { this.name = name; }
    abstract void sound(); // child MUST implement
    void sleep() { System.out.println(name + " sleeping"); } // shared concrete code
}

class Dog extends Animal {
    Dog(String name) { super(name); }
    @Override void sound() { System.out.println("Bark"); }
}

interface Flyable {
    void fly();
    default void land() { System.out.println("Landing..."); } // Java 8+ default method
}

class Bird extends Animal implements Flyable {
    Bird(String name) { super(name); }
    @Override void sound() { System.out.println("Chirp"); }
    @Override public void fly() { System.out.println("Flying high"); }
}
```

6. Overloading vs Overriding

Feature	Overloading	Overriding
Also called	Compile-time polymorphism	Runtime polymorphism
Where	Same class, same method name, different parameters	Subclass, same method signature as parent
Return type	Can be different	Must be same or covariant (subtype)
Access modifier	Can be different	Cannot be more restrictive than parent
Who decides which method runs?	Compiler at compile time	JVM at runtime based on actual object type

OVERLOADING – same class, different parameter lists:

```
print(int x)      ┌
print(String s)   ┤ Compiler picks the right one based on argument types
print(int x, int y) ┘
```

OVERRIDING – child class replaces parent's method:

Animal.speak() → Dog.speak() (replaces at runtime)

```
Animal animal = new Dog();
animal.speak(); // prints "Bark" – JVM uses Dog's version, not Animal's
```

Code Example

```
// OVERLOADING – same class, different parameters
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}

// OVERRIDING – child replaces parent method
class Animal {
    void speak() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    @Override
    void speak() { System.out.println("Bark"); } // replaces parent's speak()
}

Animal animal = new Dog();
animal.speak(); // Output: "Bark" – runtime decision by JVM
```

Key Interview Point

Always use `@Override` annotation when overriding. It causes a compile error if the parent method does not exist — preventing typos. Overloading requires no annotation.

7. HashMap Internals

In Simple Terms

A HashMap is like a **building with numbered mailboxes**. When you store a key-value pair, Java calculates a mailbox number from the key's hash code and puts the value there. When you look up a key, it goes directly to that mailbox — very fast on average ($O(1)$).

Internal Structure Diagram

HashMap Internal Structure (Java 8+):

```
Node<K,V>[] table;    (array of buckets – default size 16, always power of 2)

table[0] → null
table[1] → (Banana → value1)
table[2] → null
table[3] → (Apple → val) → (Apricot → val) ← collision: linked list
table[4] → ...
...
table[15]
```

When linked list in a bucket grows beyond 8 entries AND table length ≥ 64 :
→ converts to Red-Black Tree (faster search within that bucket)

Load Factor = 0.75 (default)

When size > capacity $\times$ 0.75 → array DOUBLES in size, all entries rehashed

put(key, value) — Step by Step

1. Calculate `hash` from `key.hashCode()` with bit spreading to reduce collisions
2. Find bucket index: `index = (table.length - 1) & hash`
3. If bucket is empty → insert new Node directly
4. If bucket has entries → traverse linked list/tree, compare keys with `equals()`
5. If key exists → update value. If new key → add to list/tree
6. If total size exceeds threshold → resize (double capacity) and rehash all entries

get(key) — Step by Step

1. Calculate hash of the key (same formula as put)
2. Go to the bucket at the computed index
3. Traverse the linked list or tree in that bucket
4. Compare each key using `equals()`
5. Return the value if found, or `null` if not found

Code Example

```
Map<String, Integer> fruitPrices = new HashMap<>();
fruitPrices.put("Apple", 80);    // hash("Apple") → bucket → store
fruitPrices.put("Banana", 40);
fruitPrices.get("Apple");        // hash → bucket → equals("Apple") → returns 80

// NOT thread-safe – use ConcurrentHashMap in multi-threaded code
Map<String, Integer> safeMap = new ConcurrentHashMap<>();
```

Important Interview Points

- HashMap is **not thread-safe** — use ConcurrentHashMap for shared access
- Keys should be **immutable** (like String) — if a key changes, its hash changes and you lose the entry
- Allows one `null` key and multiple `null` values
- Default initial capacity: 16. Default load factor: 0.75
- Average time complexity: $O(1)$. Worst case with trees: $O(\log n)$

8. ArrayList vs LinkedList

Operation	ArrayList	LinkedList	Winner
Internal structure	Dynamic array (resizable)	Doubly linked list of nodes	—
get(index) — random access	O(1) — direct array index	O(n) — must traverse from head	ArrayList
add at end	O(1) amortized	O(1)	Tie
add/remove in middle	O(n) — must shift elements	O(1) if you have node reference	LinkedList
Memory usage	Less overhead	More — each node stores data + 2 pointers	ArrayList
Cache performance	Better — contiguous memory	Worse — nodes scattered in memory	ArrayList
Real-world usage	~95% of all cases	Rare (specialized queues)	ArrayList

ArrayList (dynamic array):

Index: [0] [1] [2] [3] [4]

Value: [10 | 20 | 30 | 40 | _]

↑

get(2) → returns 30 instantly (direct memory access)

LinkedList (doubly linked nodes):

null ← [prev| 10 |next] ↔ [prev| 20 |next] ↔ [prev| 30 |next] → null

↑

get(2) → must walk: head → node1 → node2 (slow for large lists)

Beginner Rule

Unless you have a specific reason to use LinkedList (like implementing a Deque/Queue), always use **ArrayList**. It is faster for reading, uses less memory, and performs better with modern CPU caches.

9. volatile | 10. synchronized

9. volatile Keyword

When one thread changes a **volatile** variable, **all other threads immediately see the new value**. It fixes the **visibility** problem. It does **NOT** make operations like `count++` atomic.

WITHOUT volatile:

Thread 1: `running = false` (writes to CPU cache)
Thread 2: reads `running` → still sees `TRUE` (stale!)

WITH volatile:

Thread 1: `running = false` (flushed to main memory)
Thread 2: reads `running` → sees `FALSE` correctly

```
class Worker implements Runnable {
    private volatile boolean shutdown = false;

    public void run() {
        while (!shutdown) { /* work */ }
    }
    public void stop() {
        shutdown = true; // all threads see this
    }
}

// volatile does NOT fix this:
volatile int count = 0;
count++; // NOT atomic – use AtomicInteger!
```

10. synchronized Keyword

synchronized ensures only **one thread at a time** can execute a block or method. Other threads wait in a queue. Provides both **mutual exclusion** and **visibility**.

Shared bank account balance = 1000

Thread 1: `synchronized withdraw(600)`
→ acquires LOCK → reads 1000 → writes 400 → releases LOCK

Thread 2: `synchronized withdraw(600)`
→ tries LOCK → WAITS until Thread 1 releases
→ acquires LOCK → reads 400 → insufficient funds → safe!

```
class Counter {
    private int count = 0;
    public synchronized void increment() {
        count++; // now safe – only one thread at a
        time
    }
}

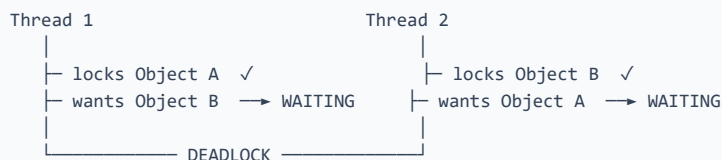
// Synchronized block – finer control
synchronized (lockObject) {
    // critical section – only one thread enters
}
```

Feature	volatile	synchronized	AtomicInteger
Visibility	Yes	Yes	Yes
Atomicity for count++	No	Yes	Yes
Blocks other threads	No	Yes	No (uses CAS)
Best for	Status flags, on/off switches	Critical sections with multiple operations	Simple counters and metrics

11. Deadlock | 12. Java Memory Model

11. Deadlock

Deadlock occurs when two or more threads are **stuck waiting for each other forever**. Each thread holds a lock the other needs. Nobody can proceed — like two people in a narrow hallway, each refusing to step back.



Four Coffman Conditions (ALL four required for deadlock):

1. Mutual exclusion — only one thread can hold a lock
2. Hold and wait — thread holds one lock while waiting for another
3. No preemption — locks cannot be forcibly taken away
4. Circular wait — A waits for B, B waits for A

How to Prevent Deadlock

1. **Lock ordering** — always acquire locks in the same global order (always A before B everywhere)
2. **tryLock with timeout** — don't wait forever; back off and retry
3. **Minimize lock scope** — hold locks for the shortest time possible
4. **Detection** — use `jstack <pid>` thread dump; shows "Found one Java-level deadlock"

12. Java Memory Model (JMM)

The JMM is a specification that defines **how threads interact through memory** — when writes by one thread become visible to another, and what instruction reordering is allowed. Without JMM rules, multi-threaded programs would be unpredictable.

Three Problems JMM Solves

- **Visibility** — Thread A sets `x=1`, but Thread B still sees `x=0` (CPU cache issue)
- **Ordering** — CPU/JVM reorders instructions for performance; Thread B sees partially constructed object
- **Atomicity** — `count++` is actually read + add + write (3 steps); two threads can lose an increment

Happens-Before Rules (Key Ones)

- Unlock of a monitor happens-before every subsequent lock of that same monitor
- Write to a volatile field happens-before every subsequent read of that volatile
- `thread.start()` happens-before any action in the started thread
- All actions in a thread happen-before `thread.join()` returns

13. Heap vs Stack vs Metaspace | 14. Garbage Collection

13. JVM Memory Areas

JVM MEMORY

STACK (per thread)

- Method call frames
- Local variables
- Object references
- Primitive locals

Auto-freed when method returns

HEAP (shared, garbage collected)

- All objects created with new()
- Arrays, Strings, instance fields
- Divided into Young + Old generation

Freed by GC when no references exist

METASPACE (Java 8+, replaced PermGen)

- Class definitions, method bytecode, static metadata
- Grows as classes are loaded; rarely garbage collected

```
class MemoryExample {
    static int classVar = 100;    // Stored in Metaspace area

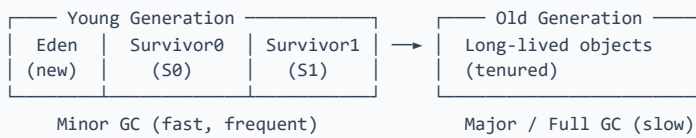
    void processOrder() {
        int quantity = 5;        // STACK – primitive local variable
        String status = "NEW";    // reference on STACK, "NEW" object on HEAP
        Order order = new Order(); // reference on STACK, Order object on HEAP
    } // method ends → stack frame removed; heap objects wait for GC
}
```

Memory Area	What It Stores	Lifetime	Garbage Collected?
Stack	Method frames, local variables, object references	Duration of method call	No — auto-removed when method returns
Heap	All objects (new), arrays, instance fields	Until no references point to them	Yes — by Garbage Collector
Metaspace	Class structures, static metadata, bytecode	While class is loaded in JVM	Rarely (class unloading)

14. Garbage Collection (GC)

Garbage Collection **automatically frees memory** occupied by objects that are no longer reachable (no active references). You never call `delete` in Java — the GC handles it for you.

HEAP divided into generations:



Object lifecycle:

new Object() → created in Eden
Minor GC → if still alive, moved to Survivor space
More GC cycles → promoted to Old Generation
No references → eligible for GC → memory reclaimed

GC Type	What It Cleans	Speed	When It Runs
Minor GC	Young generation (Eden + Survivors)	Fast (milliseconds)	Very frequently
Major GC	Old generation	Slow (can be seconds)	When old gen is full
Full GC	Entire heap	Slowest — pauses application	When heap is nearly full
G1 GC (Java 9+ default)	Regions across entire heap	Targets predictable pause times	Balanced default choice

15. LRU Cache Design

In Simple Terms

LRU = Least Recently Used. A cache stores a limited number of items for fast access. When the cache is full and a new item arrives, the item that has **not been accessed for the longest time** is removed. Like a small desk — when it is full, you remove the paper you have not looked at in the longest time.

In Simple Terms

LRU = Least Recently Used. A cache stores a limited number of items for fast access. When the cache is full and a new item arrives, the item that has **not been accessed for the longest time** is removed. Like a small desk — when it is full, you remove the paper you have not looked at in the longest time.

```

put(A) → Cache: [ A ]
put(B) → Cache: [ A, B ]
put(C) → Cache: [ A, B, C ]      ← cache is now FULL

get(A) → Cache: [ B, C, A ]      ← A moved to "most recently used" position

put(D) → Cache: [ C, A, D ]      ← B was evicted (least recently used)

```

```

MRU (Most Recently Used) head → LRU (evict this) tail

```

```
class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    LRUCache(int capacity) {
        super(capacity, 0.75f, true); // true = access-order mode (LRU behavior)
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity; // auto-remove oldest when over capacity
    }
}

// Demo
LRUCache<Integer, String> cache = new LRUCache<>(3);
cache.put(1, "A");
cache.put(2, "B");
cache.put(3, "C");
cache.get(1); // 1 becomes most recently used
cache.put(4, "D"); // evicts key 2 (least recently used)
```

Approach 2: HashMap + Doubly Linked List (Interview Favorite)

Why this design achieves $O(1)$ for both get and put:

HashMap: key $\rightarrow$ Node reference ($O(1)$ lookup by key)
Doubly Linked List: HEAD(MRU) $\longleftrightarrow$ Node $\longleftrightarrow$ Node $\longleftrightarrow$ TAIL(LRU)

get(key):

1. HashMap lookup $\rightarrow$ find Node $O(1)$
2. Remove node from current position $O(1)$
3. Insert node at HEAD (most recent) $O(1)$

put(key, value):

- If key exists $\rightarrow$ update value, move to HEAD $O(1)$
- If new + full $\rightarrow$ remove TAIL node, add at HEAD $O(1)$
- If new + space $\rightarrow$ add at HEAD $O(1)$

Total: get $O(1)$, put $O(1)$, Space $O(\text{capacity})$

Interview Question: Design an LRU Cache with $O(1)$ get and put.

Answer: Use HashMap for $O(1)$ key lookup + Doubly Linked List for $O(1)$ move-to-front and $O(1)$ eviction from tail. HashMap stores key $\rightarrow$ Node mapping. DLL maintains access order with HEAD=most recent, TAIL=least recent.

16. Quick Revision Cheat Sheet

#	Topic	One-Line Answer	Interview Must-Know
1	== vs equals	== compares memory address; equals compares content	Always use equals() for objects
2	hashCode + equals	Equal objects must have same hashCode	Override both; use same fields
3	String / Builder / Buffer	Immutable / fast mutable / thread-safe mutable	StringBuilder in loops
4	Immutability	Cannot change after creation	final class + final fields + no setters
5	Abstract vs Interface	One inheritance with shared code vs multiple contracts	Interface = "can do"; Abstract = "is-a"
6	Overload vs Override	Same class different params vs child replaces parent	Compile-time vs runtime polymorphism
7	HashMap	Array of buckets + hashCode + equals	Not thread-safe; load factor 0.75
8	ArrayList vs LinkedList	Array = fast random access; Linked = fast middle insert	Use ArrayList 95% of the time
9	volatile	Visibility across threads, not atomicity	Good for flags; not for count++
10	synchronized	One thread at a time + visibility	Keep lock scope small
11	Deadlock	Circular waiting for locks	Consistent lock ordering
12	JMM	Rules for thread memory visibility	happens-before; volatile/sync/Atomic
13	Stack / Heap / Metaspace	Local variables / objects / class metadata	new() goes to heap; locals to stack
14	Garbage Collection	Auto-deletes unreachable objects	Young gen = minor GC; Old = major GC
15	LRU Cache	Evict least recently used when full	HashMap + Doubly Linked List = O(1)